

ADR-02 — Centralización del acceso a la API en el frontend

Estado

Propuesto

Contexto

Durante el análisis del frontend del sistema **ERP Iglesias**, desarrollado con **Angular**, se identificó que múltiples componentes pueden realizar llamadas HTTP directamente hacia la API del backend.

Cuando cada componente gestiona sus propias peticiones HTTP, se generan varios problemas arquitectónicos:

- duplicación de lógica para consumir la API
- dificultad para mantener las llamadas HTTP
- dispersión de la lógica de comunicación con el backend
- mayor acoplamiento entre componentes y servicios HTTP

Situación actual posible:

```
PeopleComponent → llama API  
CoursesComponent → llama API  
PaymentsComponent → llama API
```

En este enfoque cada componente implementa su propia lógica de comunicación con el backend.

Problema

Cuando los componentes realizan llamadas HTTP directamente:

- se repite código para consumir la API
- es más difícil modificar endpoints globalmente
- la lógica de acceso a la API queda distribuida en múltiples lugares
- se reduce la mantenibilidad del frontend

Esto genera **duplicación de código y mayor acoplamiento entre componentes y servicios HTTP**.

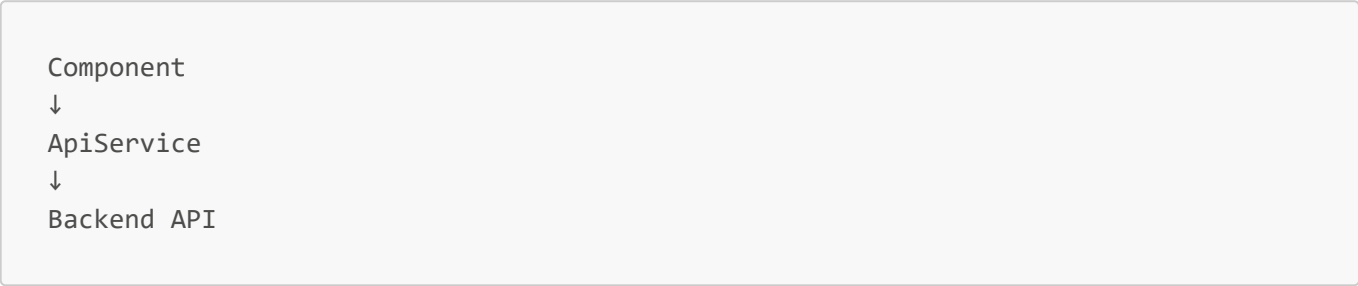
Decisión Arquitectónica

Se propone **centralizar todas las llamadas a la API en un servicio común del frontend**, por ejemplo:

```
api.service.ts
```

Los componentes del frontend no deberán realizar llamadas HTTP directamente, sino delegar estas operaciones al servicio de API.

Arquitectura propuesta:



Este servicio será responsable de gestionar todas las solicitudes HTTP hacia el backend.

Patrón de Diseño Aplicado

Service Pattern

Este patrón permite encapsular la lógica de comunicación con servicios externos dentro de una clase especializada.

Principio SOLID Aplicado

Single Responsibility Principle (SRP)

Cada componente tendrá una responsabilidad clara:

Component → manejar la lógica de interfaz de usuario ApiService → manejar la comunicación con la API

Cambio Concreto Propuesto

Se propone crear un servicio central para gestionar las llamadas HTTP.

Ejemplo:

```
api.service.ts
```

Este servicio contendrá métodos para interactuar con los diferentes endpoints del backend.

Ejemplo de métodos:

```
getPeople()
createPerson()
getCourses()
getPayments()
```

Los componentes utilizarán este servicio para comunicarse con la API.

Código Actual (ANTES)

Un componente podría realizar llamadas HTTP directamente:

```
export class PeopleComponent {

  constructor(private http: HttpClient) {}

  loadPeople() {
    this.http.get('/api/people')
      .subscribe(data => {
        console.log(data);
      });
  }
}
```

Problemas:

- lógica HTTP dentro del componente
- duplicación de código en otros componentes

Mejora Propuesta (DESPUÉS)

Se crea un servicio central para gestionar las llamadas HTTP.

ApiService

```
@Injectable({
  providedIn: 'root'
})
export class ApiService {
```

```
constructor(private http: HttpClient) {}

getPeople() {
  return this.http.get('/api/people');
}

getCourses() {
  return this.http.get('/api/courses');
}
}
```

Componente utilizando el servicio

```
export class PeopleComponent {

  constructor(private apiService: ApiService) {}

  loadPeople() {
    this.apiService.getPeople()
      .subscribe(data => {
        console.log(data);
      });
  }
}
```

Arquitectura resultante:

```
Component
↓
ApiService
↓
Backend API
```

Beneficios Arquitectónicos

La centralización del acceso a la API proporciona:

- reducción de duplicación de código
- mejor organización del frontend
- desacoplamiento entre componentes y llamadas HTTP
- mayor mantenibilidad del sistema
- facilidad para modificar endpoints en un solo lugar

Impacto en el Sistema

El cambio no modifica la funcionalidad del sistema.

Los componentes continúan consumiendo la API del backend, pero ahora lo hacen a través de un servicio centralizado.

Esto mejora la arquitectura del frontend sin afectar el comportamiento de la aplicación.

Resultado Arquitectónico

Arquitectura final propuesta:

```
graph TD
    Component --> ApiService
    ApiService --> Backend_API[Backend API]
```

Esta estructura mejora la organización del frontend, facilita el mantenimiento del código y permite aplicar buenas prácticas de desarrollo en aplicaciones Angular.